

Given Prompt:

Generate C code to implement the following requirements:

Objective

Write a C program that picks a secret number between 1 and 10.

The user gets up to 3 tries to guess the number.

Requirements • Use a fixed secret number (for example, 7).

- Use printf to prompt the user and scanf to read their guess.

- Use a while or for loop to let the user guess up to 3 times.

- Use if-else statements to check if the guess is too high, too low, or correct.

- Stop the loop immediately if the user guesses the correct number.

- Print a final win or lose message.

Example Output

Guess a number between 1 and 10.

Attempt 1/3. Enter your guess: 5

Too low! Try again.

Attempt 2/3. Enter your guess: 8

Too high! Try again.

Attempt 3/3. Enter your guess: 7

Correct! You win!

Gemini Code:

```
#include <stdio.h>
```

```
int main(void) {
```

```
    const int SECRET_NUMBER = 7;
```

```
    const int MAX_ATTEMPTS = 3;
```

```
    int guess = 0;
```

```
    int won = 0;
```

```
    printf("Guess a number between 1 and 10.\n");
```

```
    for (int attempt = 1; attempt <= MAX_ATTEMPTS; attempt++) {
```

```
        printf("Attempt %d/%d. Enter your guess: ", attempt, MAX_ATTEMPTS);
```

```
        if (scanf("%d", &guess) != 1) {
```

```
            printf("Invalid input. Please enter a number.\n");
```

```
            break;
```

```
        }
```

```
        if (guess == SECRET_NUMBER) {
```

```

        printf("Correct! You win!\n");
        won = 1;
        break; // Stop the loop immediately on correct guess
    } else if (guess < SECRET_NUMBER) {
        printf("Too low! Try again.\n");
    } else {
        printf("Too high! Try again.\n");
    }
}

if (!won) {
    printf("Game over! You've used all attempts. The secret number was %d.\n", SECRET_NUMBER);
}

return 0;
}
=====

```

Analysis:

Correctness:

- Execution: gives desired output
- Edge cases: code does have some errors for invalid inputs, but it also immediately ends the program after receiving one, stating that all attempts have been used up.
- Error handling: errors are clearly labeled and there are no crashes

Time Complexity:

- It uses a for loop primarily, the faster of the two types typically

Space Complexity

- Gemini used 4 variables and a for loop. Not terrible but certain variables like “won” aren’t necessary for what i need it for.

Maintanability:

- Comments: almost not present. Outlines the break upon a correct guess but no other explanations were given.
- Context: variable names are clear and outline their usage well.

Codellama code:

```

#include <stdio.h>

int main() {
    int guess;
    int secretNumber = 7;
    int tries = 3;

    printf("Guess a number between 1 and 10.\n");
}

```

```

while (tries > 0) {
    scanf("%d", &guess);

    if (guess == secretNumber) {
        printf("Correct! You win!\n");
        break;
    } else if (guess < secretNumber) {
        printf("Too low! Try again.\n");
    } else {
        printf("Too high! Try again.\n");
    }

    tries--;
}

if (tries == 0) {
    printf("You lose...\n");
}

return 0;
}

```

Analysis:

Correctness:

- Execution: gives desired output
- Edge cases: completely breaks with a string input. Just runs through the remaining guesses while saying "Too high!"
- Error handling: no error messages at all

Time Complexity:

- Uses a while loop, typically slower than a for loop

Space Complexity

- Uses 3 variables with all of them necessary.

Maintanability:

- Comments: None provided.
- Context: variable names are clear and state purpose.

Conclusion:

For my two AIs I chose gemini, sourced from Google's dedicated website, and Codellama, sourced from Ollama. Between the two of them Gemini was far better as a start point. Gemini seemed like it was going for a finished file, whereas Codellama tried to write as few lines as possible. Because of that Codellama had no comments, error messages, and lacked both speed and clarity.

I improved gemini's error clarity by adjusting the language so that it is less jarring when the program ends. I removed all constant variables to save space, added extensive comments

and logs explaining the purpose of each line, and reorganized the if conditions for easier readability.